МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«ОРГАНІЗАЦІЯ БАЗ ДАНИХ»

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 4

Виконав:
студент групи КН-25-1
Рафієв Б. С.
Перевірив:
асистент кафедри АІС
Андреєв П.І.

Кременчук 2026

Тема: **Створення вкладених запитів.**

Мета: навчитися створювати вкладені запити до таблиць бази даних.

Порядок виконання роботи

Лабораторну роботу виконано в середовищі Microsoft SQL Server Management Studio 22. Для запитів першої частини використано навчальну базу даних Sales111, створену під час лабораторної роботи № 2; для індивідуального завдання – базу даних LibraryDB (варіант № 1 «Бібліотека»). Усі запити виконано через вікно New Query, результати наведено на відповідних рисунках. Для перевірки запитів 1.13 і 1.14 до таблиці Customers додано замовника з кодом 2005 (Шевченко), який ще не зробив жодного замовлення.

1. Запити до навчальної бази даних Sales111

Нижче наведено запити з підзапитами, передбачені завданням, та результати їх виконання. У кожному запиті підзапит виділено дужками; для наочності псевдоніми стовпців задано українською мовою.

1.1. Визначити дати замовлень, що перевищили за сумою середнє значення, і вказати для цих замовлень перевищення над середнім рівнем.

```
SELECT DATE_OD AS Дата_замовлення,
       AMOUNT_OD AS Сума,
       CAST(AMOUNT_OD - (SELECT AVG(AMOUNT_OD) FROM Orders)
            AS DECIMAL(10, 2)) AS Перевищення
FROM Orders
WHERE AMOUNT_OD > (SELECT AVG(AMOUNT_OD) FROM Orders)
ORDER BY DATE_OD, AMOUNT_OD;
```

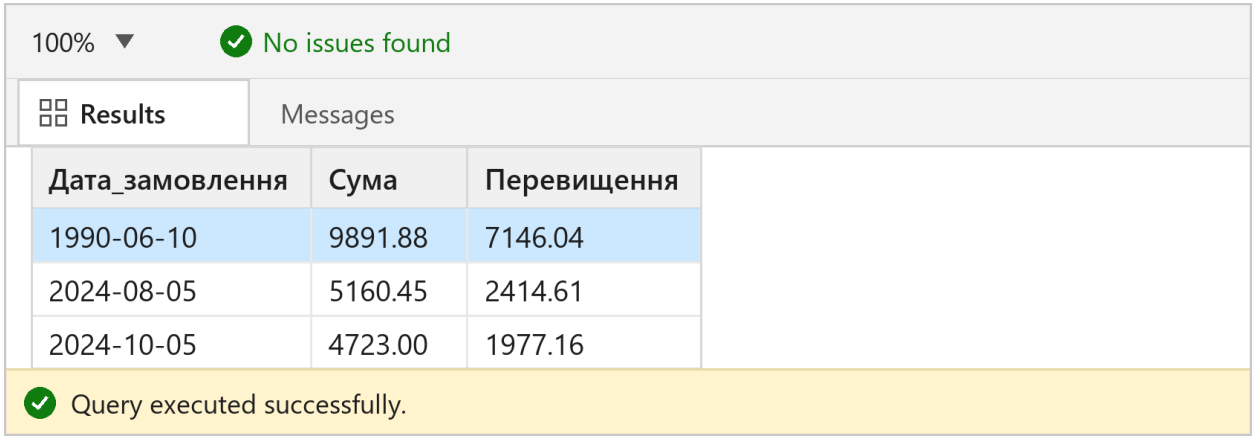


Рисунок 1 – Результат виконання запиту 1.1

Скалярний підзапит (SELECT AVG(AMOUNT_OD) FROM Orders) обчислює середню суму замовлення один раз і використовується як у списку стовпців, так і в умові WHERE.

1.2. Визначити ім'я продавця, у якого було зроблено найбільше за сумою замовлення.

```
SELECT NAME_SP AS Продавець
FROM Salespeople
WHERE ID_SP = (SELECT TOP 1 ID_SP
               FROM Orders
               ORDER BY AMOUNT_OD DESC);
```

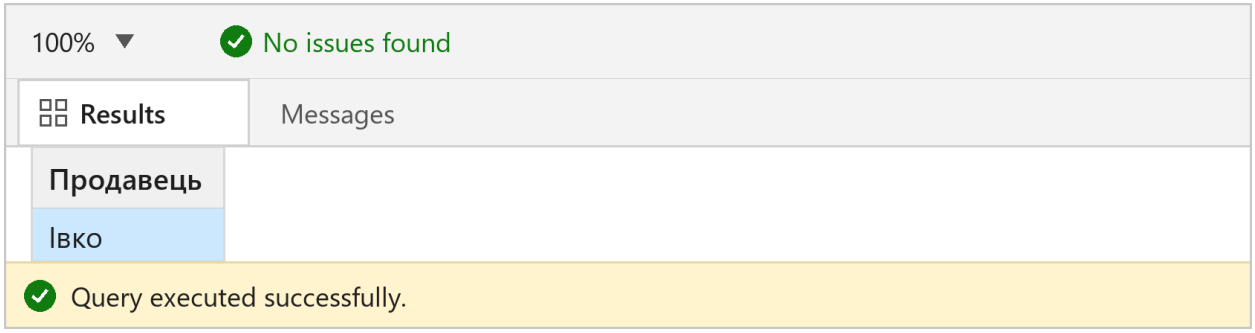


Рисунок 2 – Результат виконання запиту 1.2

Підзапит повертає код продавця, який оформив замовлення з максимальною сумою. Оператор TOP 1 з ORDER BY ... DESC забезпечує єдине скалярне значення.

1.3. Визначити замовників, у замовленнях яких сума відрізняється від максимальної не більш ніж на 10 %.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       o.AMOUNT_OD AS Сума
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.AMOUNT_OD >= (SELECT MAX(AMOUNT_OD) FROM Orders) * 0.9;
```

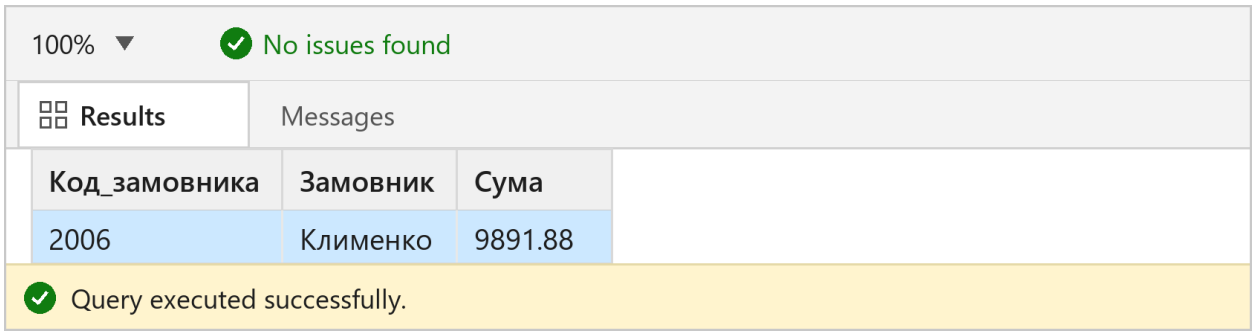


Рисунок 3 – Результат виконання запиту 1.3

Максимальна сума замовлення визначається скалярним підзапитом, після чого від неї обчислюється межа 90 % (максимум мінус 10 %).

1.4. Визначити дати, коли середня сума замовлення за день виявилася більшою за середній показник за всіма замовленнями взагалі.

```
SELECT DATE_OD AS Дата,
       COUNT(*) AS Кількість_замовлень,
       CAST(AVG(AMOUNT_OD) AS DECIMAL(10, 2)) AS Середня_за_день
FROM Orders
GROUP BY DATE_OD
HAVING AVG(AMOUNT_OD) > (SELECT AVG(AMOUNT_OD) FROM Orders)
ORDER BY DATE_OD;
```

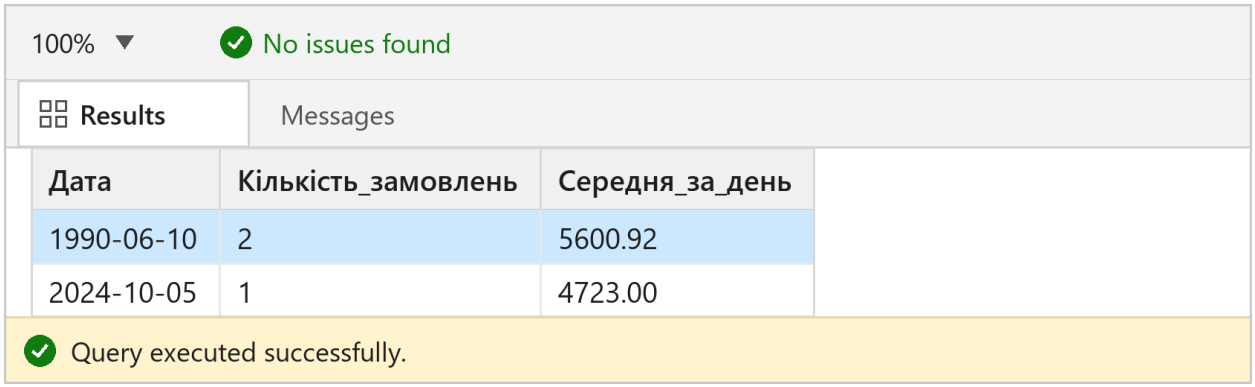


Рисунок 4 – Результат виконання запиту 1.4

Підзапит використано в умові HAVING: спочатку рядки групуються за датою, а потім відбираються лише ті дні, середня сума замовлень у яких перевищує загальний середній показник.

1.5. Визначити список продавців, що будь-коли здійснювали продажі.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець
FROM Salespeople s
WHERE EXISTS (SELECT 1
              FROM Orders o
              WHERE o.ID_SP = s.ID_SP);
```

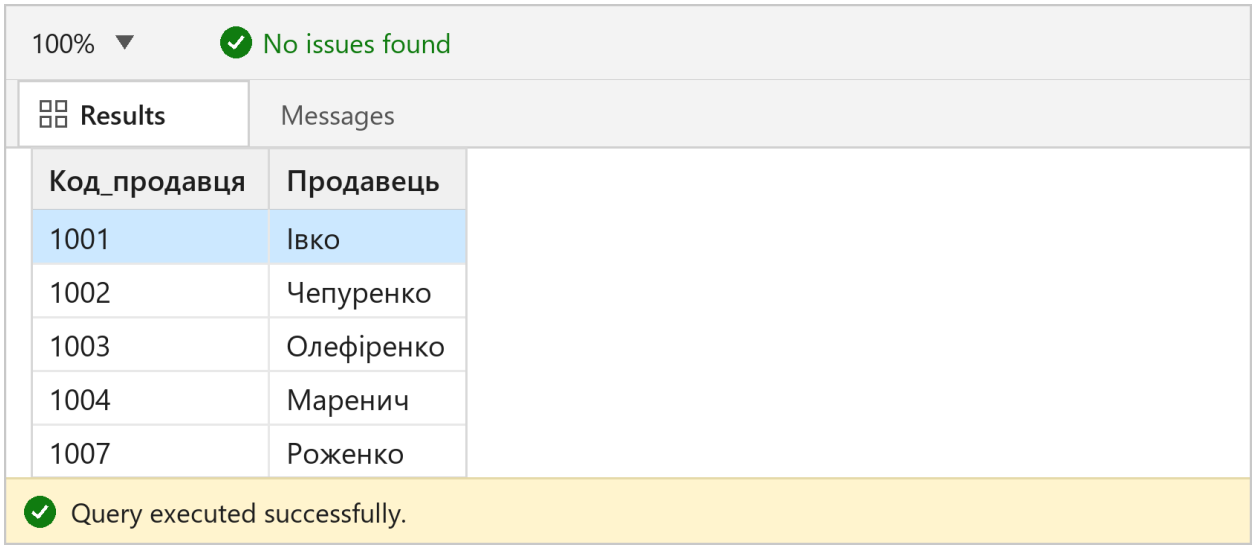
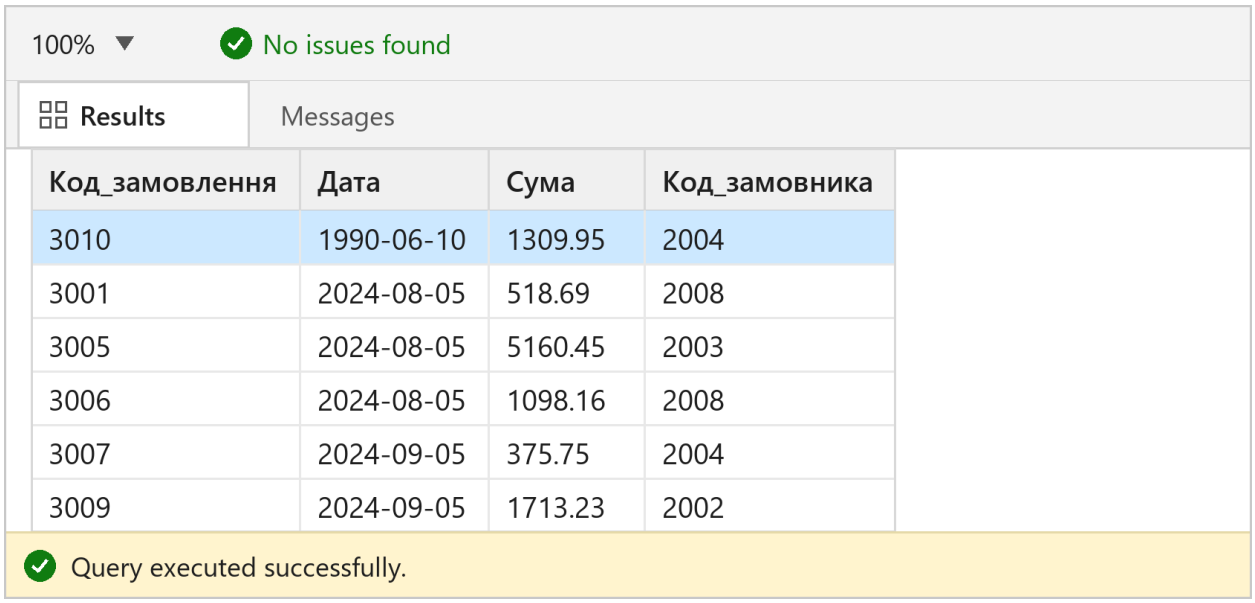


Рисунок 5 – Результат виконання запиту 1.5

Оператор EXISTS перевіряє, чи існує хоча б один рядок замовлення для кожного продавця. Підзапит є корельованим, оскільки посилається на стовпець s.ID_SP зовнішнього запиту.

1.6. Визначити замовлення, які зробили замовники, що мають рейтинг вище середнього.

```
SELECT o.ID_OD AS Код_замовлення,
       o.DATE_OD AS Дата,
       o.AMOUNT_OD AS Сума,
       o.ID_CM AS Код_замовника
FROM Orders o
WHERE o.ID_CM IN (SELECT ID_CM
                  FROM Customers
                  WHERE BONUS_CM > (SELECT AVG(BONUS_CM) FROM Customers))
ORDER BY o.DATE_OD, o.ID_OD;
```


100% No issues found

Results Messages

Код_замовлення	Дата	Сума	Код_замовника
3010	1990-06-10	1309.95	2004
3001	2024-08-05	518.69	2008
3005	2024-08-05	5160.45	2003
3006	2024-08-05	1098.16	2008
3007	2024-09-05	375.75	2004
3009	2024-09-05	1713.23	2002

Query executed successfully.

Рисунок 6 – Результат виконання запиту 1.6

Використано табличний підзапит з оператором IN та вкладеним скалярним підзапитом, що обчислює середній рейтинг замовників.

1.7. Визначити продавців, замовлення у яких здійснюють тільки замовники, що мають рейтинг вище середнього, і ніхто інший.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець
FROM Salespeople s
WHERE EXISTS (SELECT 1
              FROM Orders o
              WHERE o.ID_SP = s.ID_SP)
  AND NOT EXISTS (SELECT 1
                  FROM Orders o
                  JOIN Customers c ON c.ID_CM = o.ID_CM
                  WHERE o.ID_SP = s.ID_SP
                    AND c.BONUS_CM <= (SELECT AVG(BONUS_CM) FROM Customers));
```

100% ▼ No issues found

Results | Messages

Код_продавця	Продавець
1002	Чепуренко
1003	Олефіренко
1007	Роженко

Query executed successfully.

Рисунок 7 – Результат виконання запиту 1.7

Комбінація EXISTS і NOT EXISTS: продавець повинен мати хоча б одне замовлення, і при цьому не повинно існувати жодного його замовлення від замовника з рейтингом не вище середнього.

1.8. Визначити продавців, замовлення у яких жодного разу не зробили замовники, що мають рейтинг нижче середнього.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець
FROM Salespeople s
WHERE NOT EXISTS (SELECT 1
                  FROM Orders o
                  JOIN Customers c ON c.ID_CM = o.ID_CM
                  WHERE o.ID_SP = s.ID_SP
                    AND c.BONUS_CM < (SELECT AVG(BONUS_CM) FROM Customers));
```

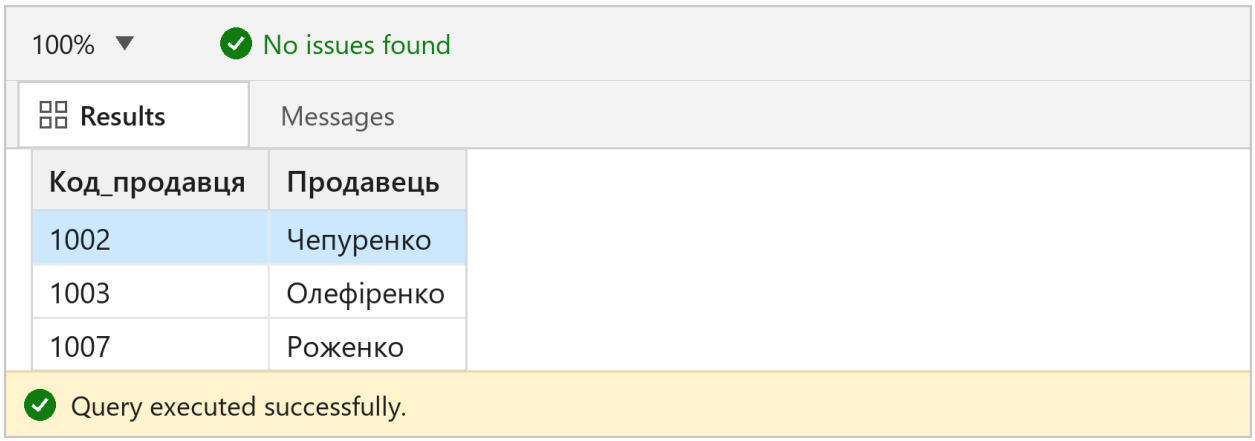


Рисунок 8 – Результат виконання запиту 1.8

На відміну від запиту 1.7, тут не вимагається наявність замовлень: продавець потрапляє до вибірки, якщо серед його замовлень немає жодного від замовника з рейтингом нижче середнього.

1.9. Визначити дати оформлення замовлень, сума яких перевищує 500 грн, замовниками, що мають рейтинг вище середнього.

```
SELECT o.DATE_OD AS Дата,
       o.AMOUNT_OD AS Сума,
       o.ID_CM AS Код_замовника
FROM Orders o
WHERE o.AMOUNT_OD > 500
  AND o.ID_CM IN (SELECT ID_CM
                  FROM Customers
                  WHERE BONUS_CM > (SELECT AVG(BONUS_CM) FROM Customers))
ORDER BY o.DATE_OD;
```

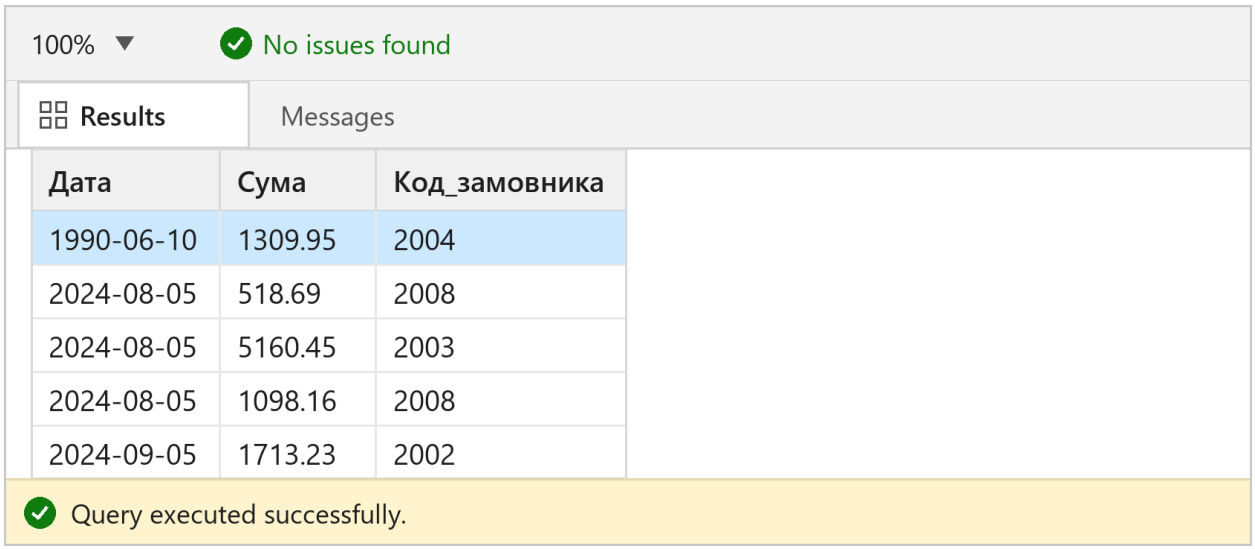


Рисунок 9 – Результат виконання запиту 1.9

Підзапит з IN відбирає замовників з рейтингом вище середнього, а зовнішній запит додатково фільтрує замовлення за сумою.

1.10. Визначити замовників, що виконали замовлення останніми.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       o.DATE_OD AS Дата
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.DATE_OD = (SELECT MAX(DATE_OD) FROM Orders);
```

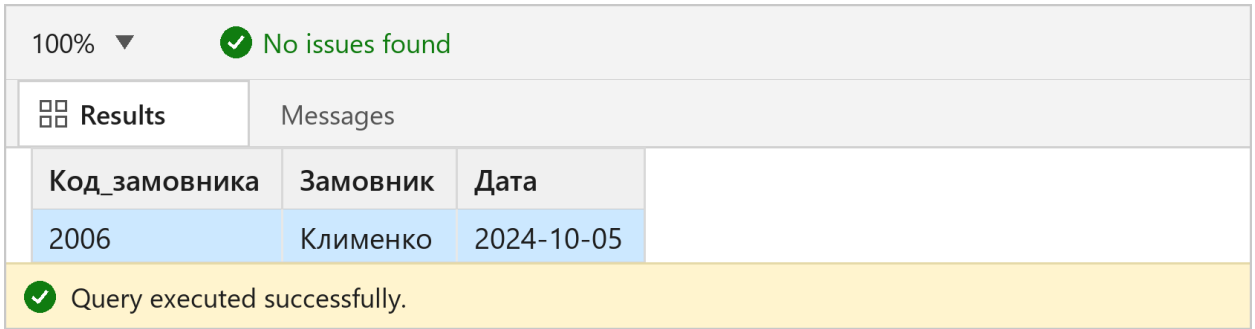


Рисунок 10 – Результат виконання запиту 1.10

Скалярний підзапит повертає найпізнішу дату замовлення, за якою і відбираються рядки зовнішнього запиту.

1.11. Знайти замовника, який зробив замовлення на найбільшу суму.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       o.AMOUNT_OD AS Сума
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.AMOUNT_OD = (SELECT MAX(AMOUNT_OD) FROM Orders);
```

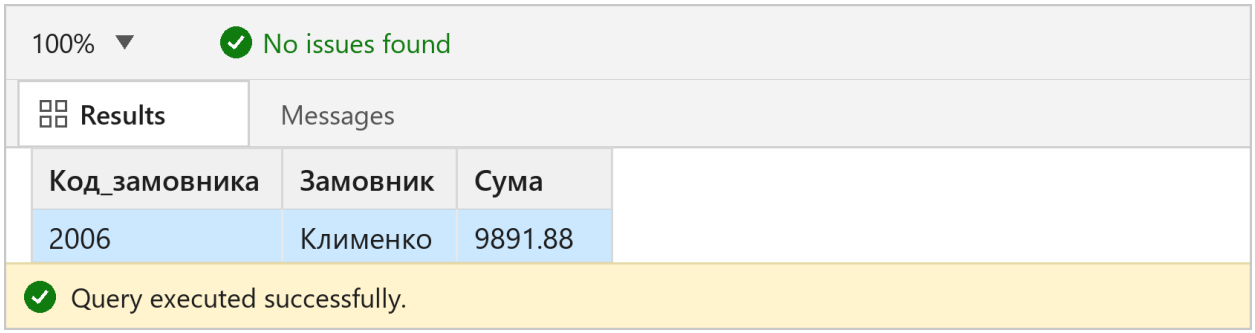


Рисунок 11 – Результат виконання запиту 1.11

Порівняння з результатом скалярного підзапиту MAX дозволяє знайти замовлення з максимальною сумою, а з'єднання з Customers – прізвище замовника.

1.12. Знайти замовників, сума в замовленнях яких перевищує такий самий показник хоча б в одному замовленні тих замовників, що мають рейтинг вище середнього.

```
SELECT DISTINCT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.AMOUNT_OD > ANY (SELECT o2.AMOUNT_OD
                         FROM Orders o2
                         WHERE o2.ID_CM IN (SELECT ID_CM
                                            FROM Customers
                                            WHERE BONUS_CM > (SELECT AVG(BONUS_CM)
                                                              FROM Customers)));
```

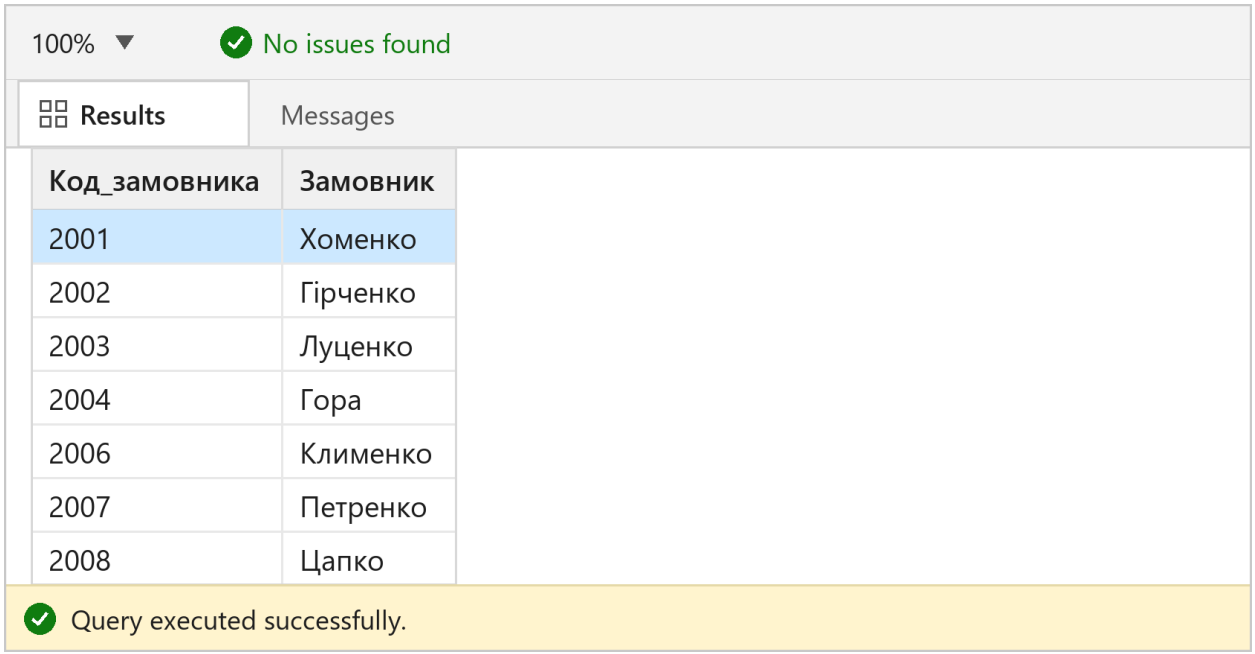


Рисунок 12 – Результат виконання запиту 1.12

Оператор ANY означає, що умова має виконуватися хоча б для одного значення з набору, який повертає підзапит (еквівалентно > MIN(...)). Використано три рівні вкладеності підзапитів.

1.13. Визначити замовників, що зробили хоча б одне замовлення.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник
FROM Customers c
WHERE EXISTS (SELECT 1
              FROM Orders o
              WHERE o.ID_CM = c.ID_CM)
ORDER BY c.ID_CM;
```

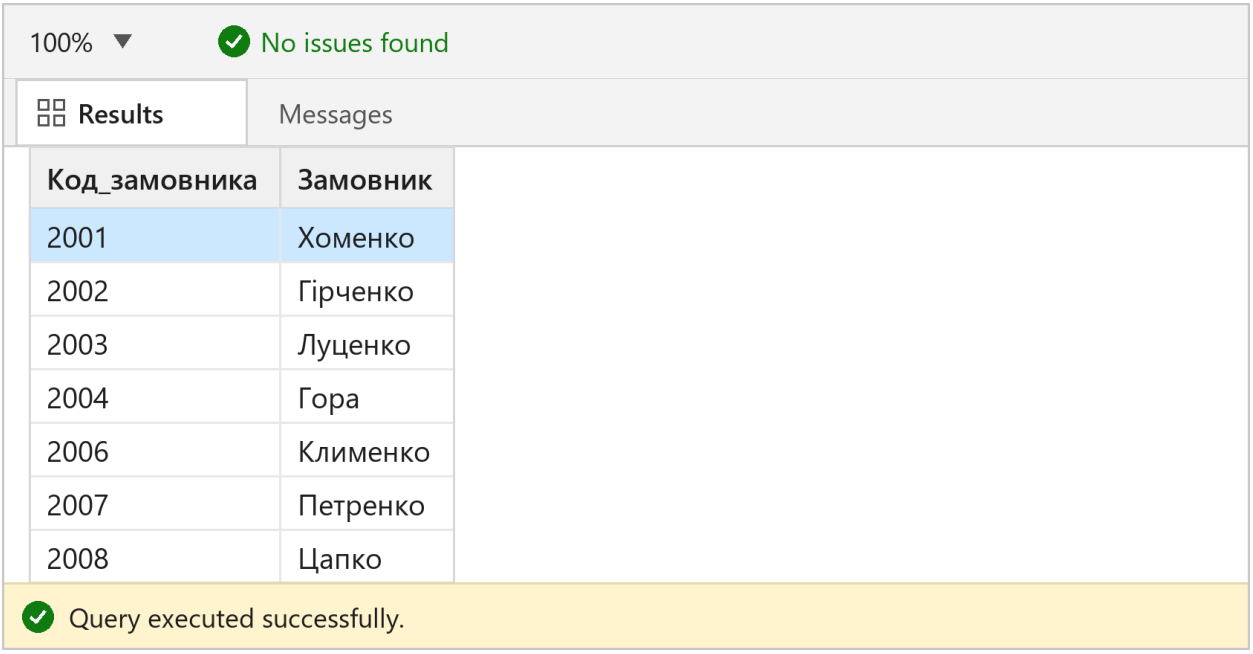


Рисунок 13 – Результат виконання запиту 1.13

Корельований підзапит з EXISTS виконується для кожного замовника і повертає TRUE, якщо в таблиці Orders є хоча б один його рядок.

1.14. Визначити замовників, які ще не зробили жодного замовлення.

```
SELECT c.ID_CM AS Код_замовника,
       c.NAME_CM AS Замовник,
       c.CITY_CM AS Місто,
       c.BONUS_CM AS Рейтинг
FROM Customers c
WHERE NOT EXISTS (SELECT 1
                  FROM Orders o
                  WHERE o.ID_CM = c.ID_CM);
```

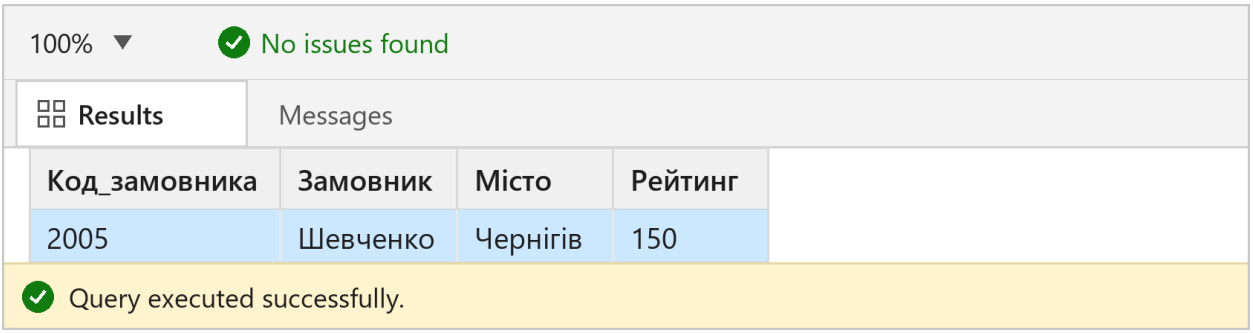


Рисунок 14 – Результат виконання запиту 1.14

Оператор NOT EXISTS повертає замовників, для яких підзапит не знайшов жодного замовлення. Запит 1.13 і 1.14 дають взаємодоповнювальні результати.

1.15. Визначити продавців, що виконали найбільше за сумою замовлення, серед продавців з рейтингом нижче середнього.

```
SELECT s.ID_SP AS Код_продавця,
       s.NAME_SP AS Продавець,
       SUM(o.AMOUNT_OD) AS Сума_замовлень
FROM Salespeople s
JOIN Orders o ON o.ID_SP = s.ID_SP
WHERE s.COMM_SP < (SELECT AVG(COMM_SP) FROM Salespeople)
GROUP BY s.ID_SP, s.NAME_SP
HAVING SUM(o.AMOUNT_OD) = (
    SELECT MAX(t.Сума)
    FROM (SELECT SUM(o2.AMOUNT_OD) AS Сума
          FROM Orders o2
          JOIN Salespeople s2 ON s2.ID_SP = o2.ID_SP
          WHERE s2.COMM_SP < (SELECT AVG(COMM_SP)
                              FROM Salespeople)
          GROUP BY s2.ID_SP) AS t);
```

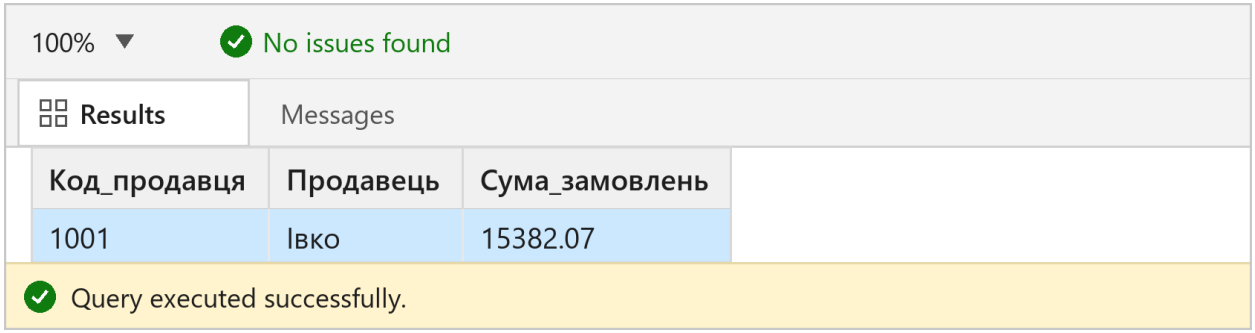


Рисунок 15 – Результат виконання запиту 1.15

У таблиці Salespeople немає окремого стовпця рейтингу, тому як рейтинг використано комісійні продавця (COMM_SP). Підзапит у FROM (похідна таблиця) обчислює суми замовлень кожного продавця, а зовнішній MAX вибирає найбільшу з них.

2. Індивідуальне завдання. База даних LibraryDB

Індивідуальне завдання виконано для варіанта № 1 – предметна область «Бібліотека». Використано створену в лабораторній роботі № 2 базу даних LibraryDB, що містить таблиці Author, Publisher, Book, Reader і BookLoan. Складено вісім запитів із підзапитами: скалярні, табличні та корельовані підзапити, а також оператори IN, NOT IN, EXISTS, NOT EXISTS, ANY і ALL.

2.1. Книги, ціна яких перевищує середню ціну всіх книг (скалярний підзапит)

```
SELECT title AS Назва,
       price AS Ціна,
       CAST(price - (SELECT AVG(price) FROM Book) AS DECIMAL(10, 2)) AS Перевищення
FROM Book
WHERE price > (SELECT AVG(price) FROM Book)
ORDER BY price DESC;
```

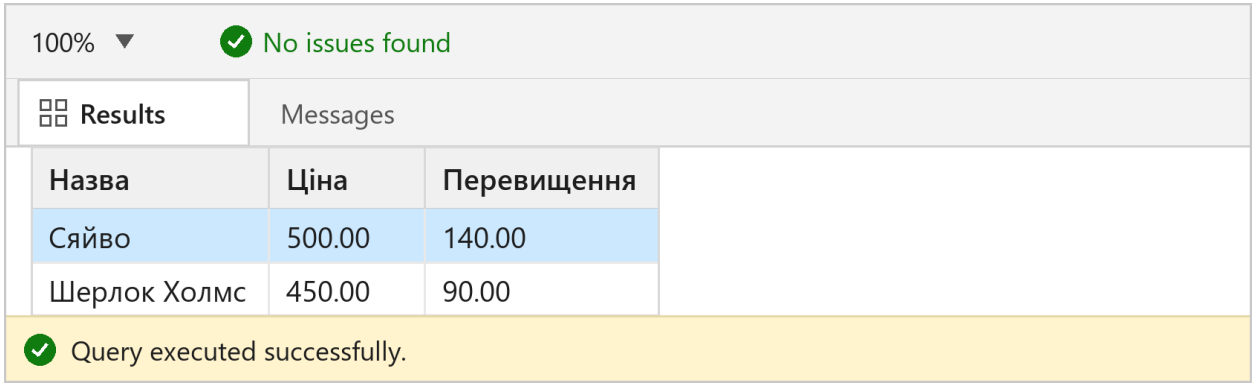


Рисунок 16 – Результат виконання запиту 2.1

Скалярний підзапит AVG(price) обчислюється один раз і використовується двічі: для відбору книг та для обчислення перевищення над середньою ціною.

2.2. Книги з максимальною ціною (скалярний підзапит з MAX)

```
SELECT id_book AS Код_книги,
       title AS Назва,
       price AS Ціна
FROM Book
WHERE price = (SELECT MAX(price) FROM Book);
```

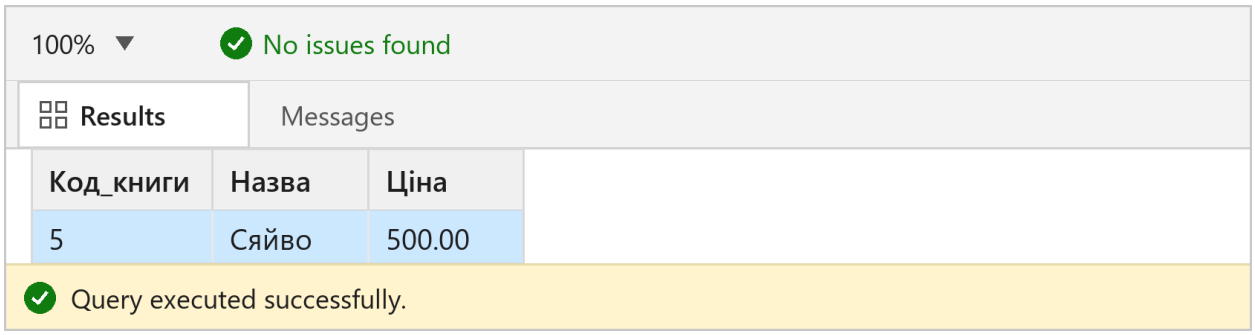


Рисунок 17 – Результат виконання запиту 2.2

Підзапит повертає єдине значення – максимальну ціну книги, яка потім порівнюється з ціною кожної книги у зовнішньому запиті.

2.3. Книги, ціна яких перевищує середню ціну книг свого видавництва (корельований підзапит)

```
SELECT b.title AS Назва,
       b.price AS Ціна,
       p.publisher_name AS Видавництво,
       CAST((SELECT AVG(b2.price)
             FROM Book b2
             WHERE b2.id_publisher = b.id_publisher)
            AS DECIMAL(10, 2)) AS Сер_ціна_вид
FROM Book b
JOIN Publisher p ON p.id_publisher = b.id_publisher
WHERE b.price > (SELECT AVG(b2.price)
                 FROM Book b2
                 WHERE b2.id_publisher = b.id_publisher)
ORDER BY p.publisher_name;
```

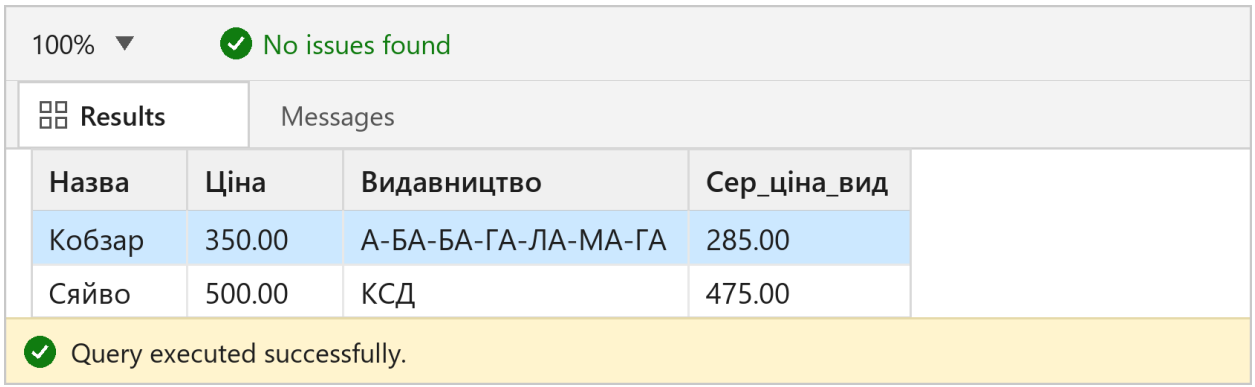


Рисунок 18 – Результат виконання запиту 2.3

Підзапит є корельованим: для кожного рядка зовнішнього запиту він заново обчислює середню ціну книг того видавництва, до якого належить поточна книга (умова b2.id_publisher = b.id_publisher).

2.4. Книги, ціна яких перевищує ціну кожної книги видавництва «А-БА-БА-ГА-ЛА-МА-ГА» (ALL)

```
SELECT b.title AS Назва,
       b.price AS Ціна
FROM Book b
WHERE b.price > ALL (SELECT b2.price
                     FROM Book b2
                     JOIN Publisher p ON p.id_publisher = b2.id_publisher
                     WHERE p.publisher_name = N'А-БА-БА-ГА-ЛА-МА-ГА');
```



Рисунок 19 – Результат виконання запиту 2.4

Оператор ALL вимагає, щоб умова виконувалася для всіх значень, які повертає підзапит, тобто ціна книги має бути більшою за найбільшу ціну книг указаного видавництва.

2.5. Книги, ціна яких перевищує ціну хоча б однієї книги видавництва «КСД» (ANY)

```
SELECT b.title AS Назва,
       b.price AS Ціна
FROM Book b
WHERE b.price > ANY (SELECT b2.price
                     FROM Book b2
                     JOIN Publisher p ON p.id_publisher = b2.id_publisher
                     WHERE p.publisher_name = N'КСД');
```

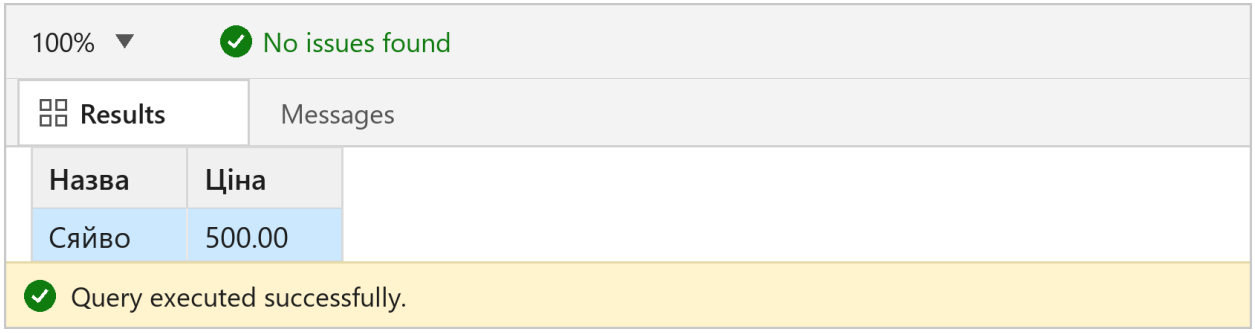


Рисунок 20 – Результат виконання запиту 2.5

Оператор ANY достатньо, щоб умова виконалася хоча б для одного значення підзапиту; результат еквівалентний умові price > MIN(...).

2.6. Автори, які мають хоча б одну книгу у фонді (EXISTS)

```
SELECT a.id_author AS Код_автора,
       a.author_name AS Автор,
       a.country AS Країна
FROM Author a
WHERE EXISTS (SELECT 1
              FROM Book b
              WHERE b.id_author = a.id_author)
ORDER BY a.author_name;
```

100% ▼ No issues found

Results | Messages

Код_автора	Автор	Країна
4	Артур Конан Дойл	Велика Британія
2	Іван Франко	Україна
3	Леся Українка	Україна
5	Стівен Кінг	США
1	Тарас Шевченко	Україна

Query executed successfully.

Рисунок 21 – Результат виконання запиту 2.6

EXISTS перевіряє лише наявність рядків у підзапиті, тому замість списку стовпців достатньо вказати SELECT 1. Підзапит корельований за id_author.

2.7. Автори, які не мають книг, виданих після 2021 року (NOT EXISTS)

```
SELECT a.id_author AS Код_автора,
       a.author_name AS Автор
FROM Author a
WHERE NOT EXISTS (SELECT 1
                  FROM Book b
                  WHERE b.id_author = a.id_author
                    AND b.year_pub > 2021)
ORDER BY a.author_name;
```

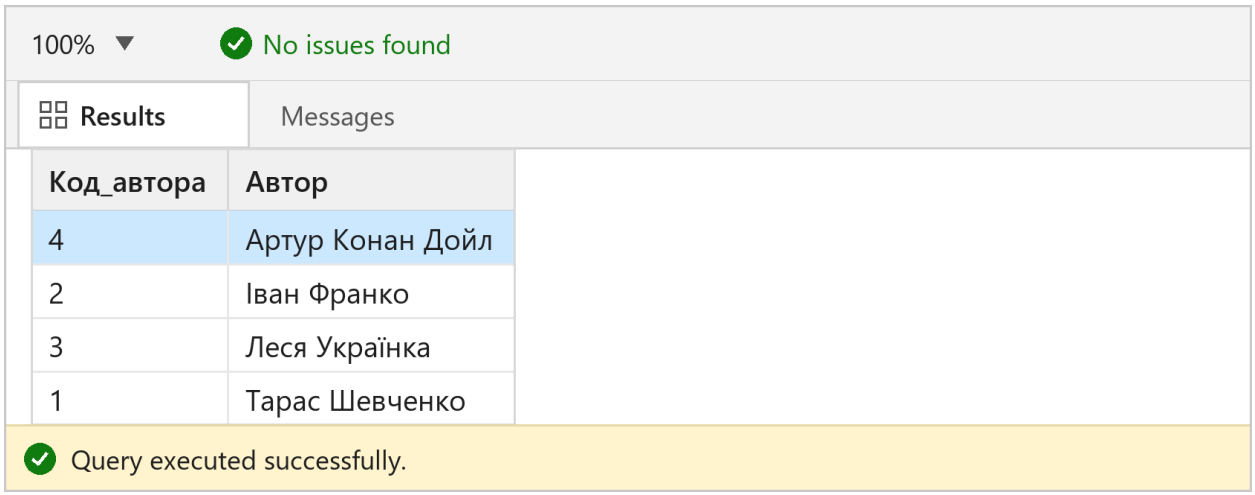


Рисунок 22 – Результат виконання запиту 2.7

NOT EXISTS повертає тих авторів, для яких підзапит не знайшов жодної книги, виданої після 2021 року.

2.8. Книги, які не видавалися читачеві з кодом 5 (NOT IN)

```
SELECT b.id_book AS Код_книги,
       b.title AS Назва,
       b.price AS Ціна
FROM Book b
WHERE b.id_book NOT IN (SELECT l.id_book
                        FROM BookLoan l
                        WHERE l.id_reader = 5)
ORDER BY b.id_book;
```

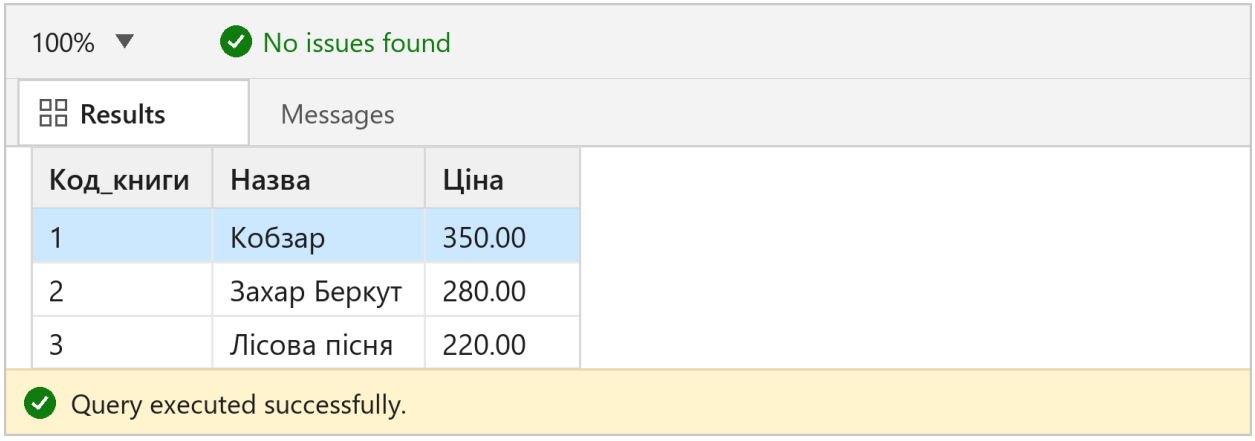


Рисунок 23 – Результат виконання запиту 2.8

Підзапит формує список книг, які видавалися читачеві з кодом 5, а NOT IN виключає їх з повного переліку книг фонду.

Відповіді на контрольні питання

1. Як виконуються вкладені запити?

Вкладений запит (підзапит) – це оператор SELECT, записаний у круглих дужках усередині іншого оператора SELECT. Підзапит може розміщуватися в умові WHERE або HAVING, у розділі FROM (похідна таблиця) та у списку стовпців SELECT. Логіка виконання така: спочатку виконується внутрішній запит, який формує проміжне (неіменоване) відношення, а потім зовнішній запит використовує його результат як звичайне значення або таблицю.

Прості (некорельовані) підзапити не залежать від зовнішнього запиту, тому виконуються один раз, і їх результат підставляється у зовнішній запит. Корельовані підзапити посилаються на стовпці зовнішнього запиту, тому виконуються окремо для кожного рядка. Приклад простого підзапиту (запит 1.11):

```
SELECT c.NAME_CM, o.AMOUNT_OD
FROM Orders o JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.AMOUNT_OD = (SELECT MAX(AMOUNT_OD) FROM Orders);
```

Тут підзапит MAX(AMOUNT_OD) обчислюється один раз, а потім його значення порівнюється з сумою кожного замовлення.

2. Які обмеження існують на використання підзапитів?

Під час написання підзапитів слід враховувати такі обмеження MS SQL Server:

1) підзапит обов'язково записується в круглих дужках;

2) скалярний підзапит, який використовується з операціями порівняння (=, <>, >, <, >=, <=) або у списку стовпців SELECT, повинен повертати не більше одного стовпця і не більше одного рядка, інакше виникає помилка «Subquery returned more than 1 value»;

3) підзапит після IN, ANY, ALL, EXISTS може повертати багато рядків, але для IN, ANY і ALL – рівно один стовпець; для EXISTS кількість стовпців не має значення;

4) у підзапиті не можна використовувати ORDER BY, якщо не вказано TOP, OFFSET або FOR XML; для отримання одного значення застосовують TOP 1 з ORDER BY;

5) підзапит у розділі FROM (похідна таблиця) обов'язково повинен мати псевдонім;

6) не допускається використання типів даних text, ntext та image;

7) підзапит не може містити власний оператор INTO і не може бути безпосереднім аргументом агрегатної функції (потрібно оформити його як похідну таблицю);

8) рівень вкладеності підзапитів не повинен перевищувати 32;

9) некорельований підзапит не може посилатися на стовпці зовнішнього запиту – таке посилання робить його корельованим і змінює порядок виконання.

3. Поясніть використання оператора EXISTS.

Оператор EXISTS використовується тільки разом із підзапитом і має вигляд WHERE [NOT] EXISTS (підзапит). Його результатом є логічне значення TRUE або FALSE: TRUE – якщо підзапит повернув хоча б один рядок, FALSE – якщо результуюча таблиця підзапиту порожня. Оскільки перевіряється лише факт наявності рядків, вміст списку стовпців значення не має, тому зазвичай пишуть SELECT 1 або SELECT *. Сервер припиняє обробку підзапиту після знаходження першого відповідного рядка, що робить EXISTS ефективним.

Найчастіше EXISTS застосовують із корельованими підзапитами: умова всередині підзапиту пов'язує його з поточним рядком зовнішнього запиту. Оператор NOT EXISTS дає протилежний результат і зазвичай використовується для пошуку записів, яким не відповідає жоден рядок у пов'язаній таблиці. Приклади із звіту (запити 1.13 та 1.14):

```
-- замовники, що зробили хоча б одне замовлення
SELECT c.ID_CM, c.NAME_CM
FROM Customers c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.ID_CM = c.ID_CM);

-- замовники без замовлень
SELECT c.ID_CM, c.NAME_CM
FROM Customers c
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.ID_CM = c.ID_CM);
```

4. Поясніть використання операторів ALL та ANY.

Оператори ALL і ANY (синонім SOME) застосовуються разом з операцією порівняння та підзапитом, що повертає один стовпець значень: вираз <оператор> {ALL | ANY} (підзапит). Оператор ALL вимагає, щоб умова була істинною для всіх значень, які повернув підзапит, а ANY – щоб вона була істинною хоча б для одного з них.

Зручно пам'ятати еквівалентні перетворення: x > ALL (...) те саме, що x > MAX(...); x > ANY (...) – що x > MIN(...); x = ANY (...) – що x IN (...); x <> ALL (...) – що x NOT IN (...). Якщо підзапит повертає NULL, результат порівняння стає невизначеним (UNKNOWN) і рядок не включається до вибірки. Для порожнього результату підзапиту умова з ALL вважається істинною, а з ANY – хибною. Приклади із звіту (запити 1.12, 2.4, 2.5):

```
-- сума замовлення більша хоча б за одне замовлення клієнтів
-- з рейтингом вище середнього
SELECT DISTINCT c.ID_CM, c.NAME_CM
FROM Orders o JOIN Customers c ON c.ID_CM = o.ID_CM
WHERE o.AMOUNT_OD > ANY (SELECT o2.AMOUNT_OD FROM Orders o2
                         WHERE o2.ID_CM IN (SELECT ID_CM FROM Customers
                                            WHERE BONUS_CM > (SELECT AVG(BONUS_CM)
                                                              FROM Customers)));

-- книги дорожчі за кожну книгу видавництва (ALL)
SELECT title, price FROM Book
WHERE price > ALL (SELECT price FROM Book WHERE id_publisher = 1);
```

5. Що таке корельований підзапит?

Корельований підзапит – це підзапит, який посилається на стовпці зовнішнього запиту. Через це він не може бути виконаний один раз: для кожного рядка зовнішнього запиту підзапит виконується заново з відповідними значеннями цих стовпців, і саме від них залежить його результат. У тексті запиту такий зв'язок задається псевдонімом зовнішньої таблиці, наприклад b.id_publisher у запиті 2.3.

Корельовані підзапити найчастіше використовуються з операторами EXISTS і NOT EXISTS, а також для порівняння значення рядка з агрегатом, обчисленим для його групи. На відміну від простого підзапиту, який виконується один раз для всього запиту, корельований може бути повільнішим, оскільки кількість його виконань дорівнює кількості рядків зовнішнього запиту. Приклад (запит 2.3):

```
-- книги, дорожчі за середню ціну книг свого видавництва
SELECT b.title, b.price, p.publisher_name
FROM Book b JOIN Publisher p ON p.id_publisher = b.id_publisher
WHERE b.price > (SELECT AVG(b2.price)
                 FROM Book b2
                 WHERE b2.id_publisher = b.id_publisher);
```

Тут умова b2.id_publisher = b.id_publisher пов'язує підзапит із поточним рядком зовнішнього запиту, тому для кожної книги середня ціна обчислюється окремо – для її видавництва.

Висновки:

Під час виконання лабораторної роботи № 4 було опановано створення вкладених запитів у MS SQL Server. Розглянуто скалярні та табличні підзапити, підзапити в умовах WHERE, HAVING і в розділі FROM, оператори IN, NOT IN, EXISTS, NOT EXISTS, ANY і ALL, а також корельовані підзапити. На прикладі баз Sales111 і LibraryDB виконано 15 запитів за методичними вказівками та 8 запитів за індивідуальним варіантом. Отримані результати підтверджують, що підзапити дозволяють розв'язувати задачі, які неможливо або незручно виконати одним простим запитом: порівняння з середніми та максимальними значеннями, пошук записів з наявними або відсутніми пов'язаними рядками, робота з множинами значень. Окремо надано відповіді на всі п'ять контрольних питань з методичних вказівок.